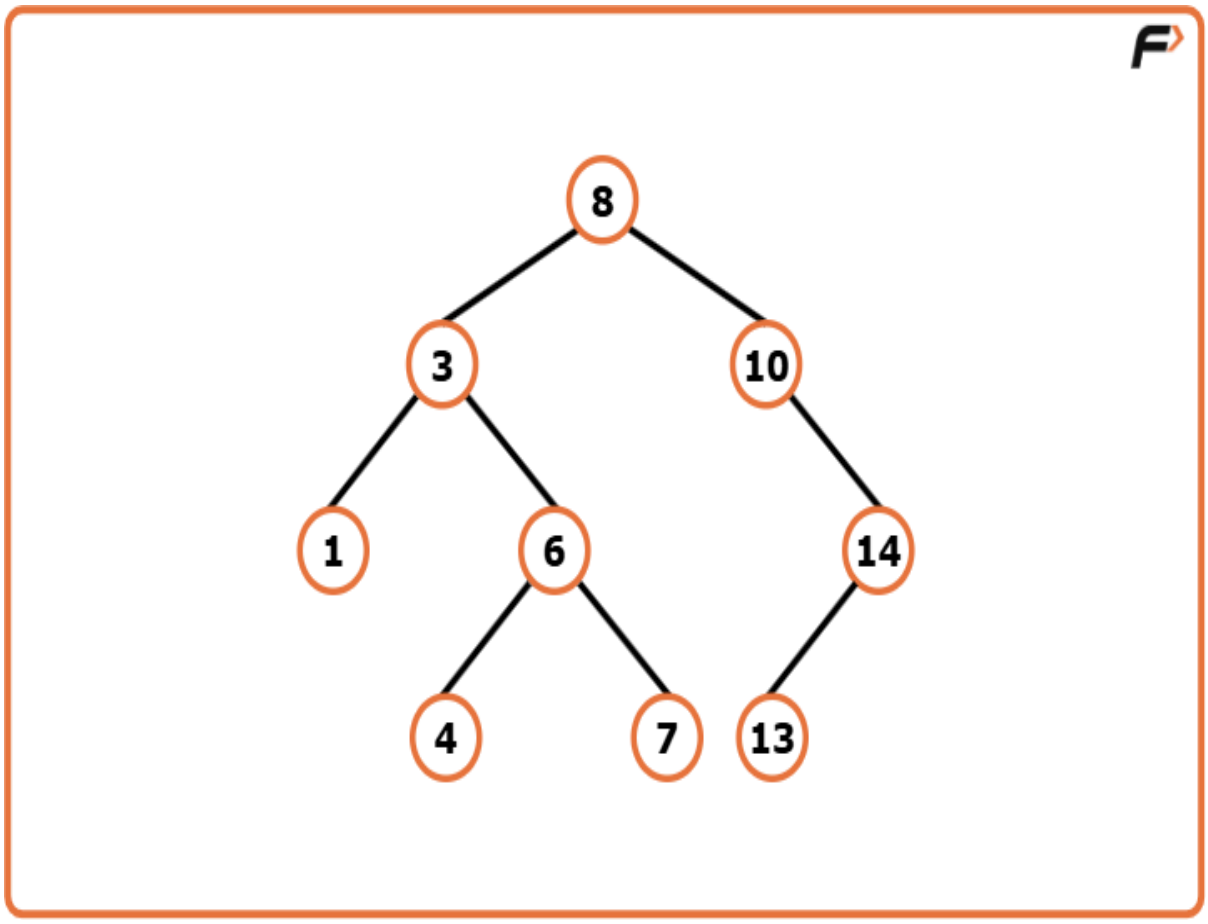
Binary Search Tree

1. Introduction to Binary Search Trees

Binary Search Trees, popularly known as BST, are the category of Binary Trees used to optimize the operation of searching an element among the Tree Nodes in a Binary Tree.

Let's understand the dynamics of a Binary Search Tree using an illustration below:

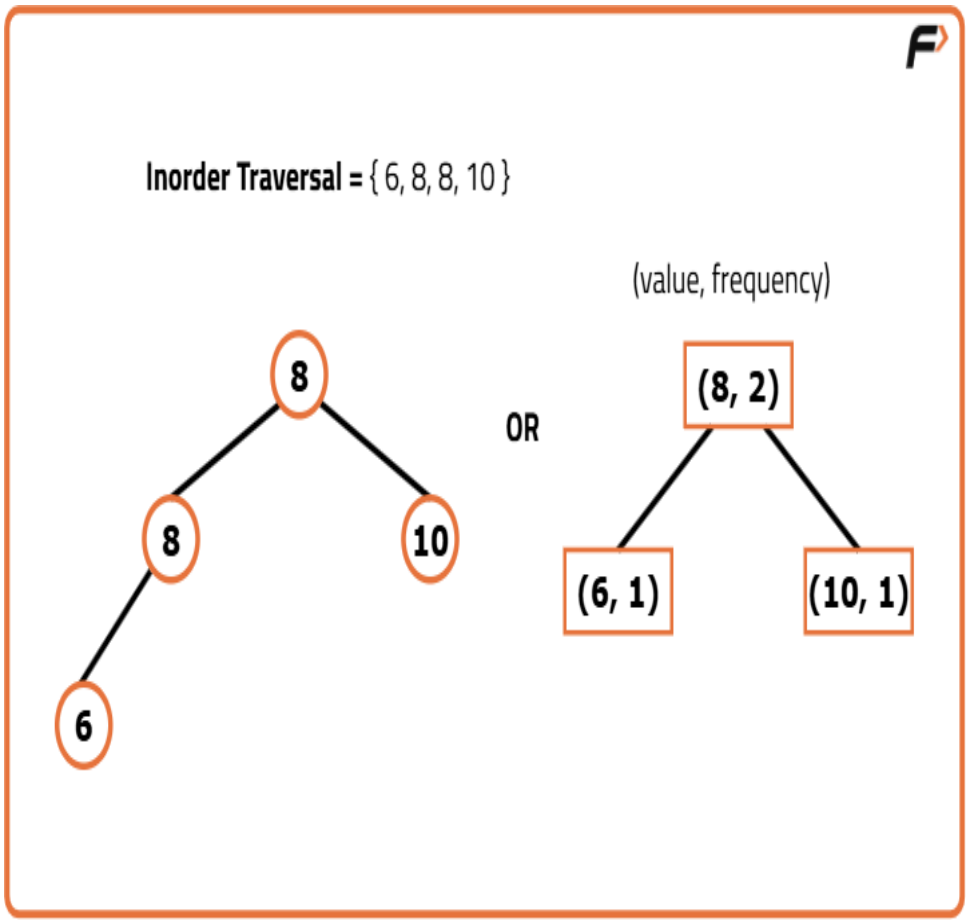


As we can visualize from the above Tree, the right subtree has all the elements greater than the root element and the left subtree has all the elements lesser than the root element. A point to note here is that **every subtree is itself a Binary Search Tree** as it contains a subtree whose root node is lesser than the value of the current node and another subtree whose root node is greater than the value of the current node.

For every given node in a Binary Search Tree if a left or a right child exists then:

```
Left Child < Node < Right Child
```

Now, for the general implementation of Binary Search Trees, **duplicate node values are not allowed**. However, in some exceptional cases of Binary Search Tree implementation, we can represent a Binary Search Tree with duplicate node values in the following ways:



Need for Binary Search Tree

After understanding how a Binary Search Tree is represented, let us now understand why is there a need to use a Binary Search Tree instead of a simple Binary Tree.

So, generally in a BST, the maximum height in almost all cases is kept in order of **log(2N)** where N = No. of nodes which is in contrast to the plain Binary Tree whose maximum height can reach the order of **N** when the tree is **skewed.**

This particularly makes the time of searching for a given node in a Binary Search Tree a lot less than searching in a simple Binary Tree.

For example, if we have to search for an element, we can directly compare it to the root node value of the BST, if the value matches then we have found the element, if say the value is greater than the root node, we say that we now move on to searching that element in the right subtree because the right subtree has all the node values greater than the root value. The search process is terminated for the left subtree as it would only search for those nodes which have values lesser than the root.

Note that, this is very similar to the Binary Search operation which we perform in Arrays to optimize the time taken to search for an element using Linear Search. Hence this category of trees is named **Binary Search Trees**.

2. Search in a Binary Search Tree

Given a **Binary Search Tree (BST)** and a key value, you need to **search for the node whose value is equal to the given key**.
If such a node exists, return that node; otherwise, return `nullptr`.

In a Binary Search Tree:

- Left subtree contains values **smaller** than the node.
- Right subtree contains values **greater** than the node.

Example Explanation

Example 1

Input BST: `8 5 12 4 7 10 14`

Key = `10`

We start from root 8:

- `10 > 8` → move right
- reach 12
- `10 < 12` → move left
- reach `10` → found

Output: `True`

Example 2

Input BST: `4 2 6 1 3 5 7`

Key = 3

If traversal ends at `nullptr` without finding the key, the value does not exist.

Output: `False`

Approach

1. Start from the root node.
2. While the current node is not `NULL`:
    - If the current node's value equals the key, return the current node.
    - If the key is smaller than the current node's value, move to the left child.
    - If the key is greater than the current node's value, move to the right child.
3. If the traversal reaches `NULL`, the key is not present. Return `nullptr`.

This works because the BST property helps us decide the direction at each step.

Code (C++)

```
#include <bits/stdc++.h>

using namespace std;



// Definition of TreeNode

struct TreeNode {

    int val;

    TreeNode* left;

    TreeNode* right;



    TreeNode(int data) {

        val = data;

        left = right = nullptr;

    }

};



class Solution {

public:
```

```
    // Function to search for a node with given key in BST
    TreeNode* searchBST(TreeNode* root, int target) {

        // Traverse until target is found or root becomes NULL
        while (root != nullptr && root->val != target) {

            // Move left if target is smaller
            if (target < root->val) {
                root = root->left;
            }
            // Move right if target is larger
            else {
                root = root->right;
            }
        }

        // Returns node if found, otherwise nullptr
        return root;
    }
};
```

```
int main() {

    TreeNode* root = new TreeNode(4);

    root->left = new TreeNode(2);

    root->right = new TreeNode(7);

    root->left->left = new TreeNode(1);

    root->left->right = new TreeNode(3);



    Solution obj;

    TreeNode* result = obj.searchBST(root, 2);



    if (result)

        cout << "Node found: " << result->val << endl;

    else

        cout << "Node not found" << endl;



    return 0;

}
```

Complexity Analysis

- **Time Complexity:**
  `O(log N)` in a balanced BST because each comparison removes half of the tree.
  `O(N)` in the worst case for a skewed tree.

- **Space Complexity:**
  `O(1)` because the solution is iterative and uses no extra data structures.

3. Find Minimum / Maximum in a Binary Search Tree

Given a **Binary Search Tree (BST)**, find:

- the **minimum value node**
- the **maximum value node**

In a BST:

- All values in the **left subtree** are smaller than the root.
- All values in the **right subtree** are greater than the root.

This property makes finding min and max very straightforward.

Question Explanation

You are given the root of a Binary Search Tree.
You need to:

- Return the node with the **minimum value**
- Return the node with the **maximum value**

If the tree is empty, return `nullptr`.

Approach

To find Minimum value:

1. Start from the root.
2. Keep moving to the **left child** because smaller values are always on the left.
3. When a node has no left child, that node is the **minimum**.

To find Maximum value:

1. Start from the root.
2. Keep moving to the **right child** because larger values are always on the right.
3. When a node has no right child, that node is the **maximum**.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) {
```

```
        val = x;
        left = right = nullptr;
    }
};

class Solution {
public:
    // Function to find minimum value node in BST
    TreeNode* findMin(TreeNode* root) {
        if (root == nullptr) return nullptr;

        while (root->left != nullptr) {
            root = root->left;
        }
        return root;
    }

    // Function to find maximum value node in BST
    TreeNode* findMax(TreeNode* root) {
        if (root == nullptr) return nullptr;

        while (root->right != nullptr) {
            root = root->right;
        }
        return root;
    }
};

int main() {
    TreeNode* root = new TreeNode(8);
    root->left = new TreeNode(3);
    root->right = new TreeNode(10);
    root->left->left = new TreeNode(1);
    root->left->right = new TreeNode(6);
    root->right->right = new TreeNode(14);

    Solution sol;
```

```
    TreeNode* minNode = sol.findMin(root);
    TreeNode* maxNode = sol.findMax(root);

    if (minNode)
        cout << "Minimum value: " << minNode->val << endl;
    if (maxNode)
        cout << "Maximum value: " << maxNode->val << endl;

    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
  `O(H)` where `H` is the height of the tree.
  - Balanced BST → `O(log N)`
  - Skewed BST → `O(N)`
- **Space Complexity:**
  `O(1)` because no recursion or extra data structures are used.

4. Ceil in a Binary Search Tree

Given a **Binary Search Tree (BST)** and a key value, you need to find the **ceiling** of that key.
The **ceiling** is defined as the **smallest value in the BST that is greater than or equal to the given key**.
If no such value exists, return `-1` (or `nullptr` logically).

Because this is a BST, we can use its ordering property to search efficiently.

Question Explanation

In a Binary Search Tree:

- Left subtree contains **smaller values**
- Right subtree contains **larger values**

While traversing the tree:

- If a node value is **equal** to the key, that value itself is the ceiling.
- If a node value is **greater** than the key, it can be a possible ceiling, but we still try to find a smaller valid one on the left.
- If a node value is **smaller** than the key, it cannot be the ceiling, so we move right.

Example Explanation

Example 1
BST: `10 5 15 2 6`
Key = 7

Traversal:

- Start at 10 → greater than 7 → possible ceil = 10 → move left
- At 5 → smaller than 7 → move right
- At 6 → smaller than 7 → move right → null

Final answer = **10**

Algorithm

1. Initialize a variable `ceil = -1`.
2. Start traversing from the root.

3. While the current node is not null:
    - If node value equals key → update `ceil` and return.
    - If key is greater than node value → move to right subtree.
    - If key is smaller than node value:
        - Update `ceil` with current node value.
        - Move to left subtree.
4. Return `ceil` after traversal ends.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) {
        val = x;
        left = right = nullptr;
    }
};

class Solution {
public:
    int findCeil(TreeNode* root, int key) {
        int ceil = -1;

        while (root != nullptr) {
            if (root->val == key) {
```

```
                ceil = root->val;
                return ceil;
            }
            if (key > root->val) {
                root = root->right;
            } else {
                ceil = root->val;
                root = root->left;
            }
        }
        return ceil;
    }
};

// Inorder traversal for verification
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

int main() {
    TreeNode* root = new TreeNode(10);
    root->left = new TreeNode(5);
    root->right = new TreeNode(15);
    root->left->left = new TreeNode(2);
    root->left->right = new TreeNode(6);

    Solution sol;
    int key = 7;

    cout << "BST Inorder: ";
    inorder(root);
    cout << endl;

    int result = sol.findCeil(root, key);
    cout << "Ceil of " << key << " is: " << result << endl;
```

```
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(H)`
  Where `H` is the height of the BST.
  - Balanced BST → `O(log N)`
  - Skewed BST → `O(N)`
- **Space Complexity:** `O(1)`
  No recursion or extra data structures are used.

5. Floor in a Binary Search Tree

Given a **Binary Search Tree (BST)** and a key value, find the **floor** of that key.
The **floor** is defined as the **largest value in the BST that is smaller than or equal to the given key**.
If such a value does not exist, return `-1`.

Question Explanation

In a Binary Search Tree:

- Left subtree contains values **smaller** than the current node.

- Right subtree contains values **greater** than the current node.

To find the floor:

- If we find the key itself, that value is the floor.
- If the key is greater than the current node, the current node is a **possible floor**, and we try to find a larger one on the right.
- If the key is smaller than the current node, the current node cannot be the floor, so we move left.

Example Explanation

Example 1
BST: `10 5 15 2 6`
Key = 7

Traversal:

- Start at 10 → 7 < 10 → move left
- At 5 → 7 > 5 → floor = 5 → move right
- At 6 → 7 > 6 → floor = 6 → move right → null

Final answer = **6**

Approach 1

Algorithm

1. Initialize a variable `floor = -1`.
2. Start traversal from the root.

3. While the current node is not null:
    - If current node value equals key, assign it to `floor` and return.
    - If key is greater than current node value:
        - Update `floor` with current node value.
        - Move to the right subtree.
    - If key is smaller than current node value:
        - Move to the left subtree.
4. Return `floor` after traversal ends.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int floorInBST(TreeNode* root, int key) {
        int floor = -1;

        while (root) {
            if (root->val == key) {
                floor = root->val;
```

```
                return floor;
            }
            if (key > root->val) {
                floor = root->val;
                root = root->right;
            } else {
                root = root->left;
            }
        }
        return floor;
    }
};

// Inorder traversal for checking BST
void printInOrder(TreeNode* root) {
    if (!root) return;
    printInOrder(root->left);
    cout << root->val << " ";
    printInOrder(root->right);
}

int main() {
    TreeNode* root = new TreeNode(10);
    root->left = new TreeNode(5);
    root->right = new TreeNode(13);
    root->left->left = new TreeNode(3);
    root->left->right = new TreeNode(6);
    root->left->right->right = new TreeNode(9);

    cout << "BST Inorder: ";
    printInOrder(root);
    cout << endl;

    Solution sol;
    int key = 8;
    int result = sol.floorInBST(root, key);

    if (result != -1)
```

```
        cout << "Floor of " << key << " is: " << result << endl;
    else
        cout << "No floor found" << endl;

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(H)`
  Where `H` is the height of the BST.
    - Balanced BST → `O(log N)`
    - Skewed BST → `O(N)`
- **Space Complexity:** `O(1)`
  The algorithm is iterative and uses constant extra space.

6. Insert a given Node in Binary Search Tree

Given a **Binary Search Tree (BST)** and a key value, insert a new node with that key into the BST.
The insertion must follow **BST properties** and **tree structure must remain valid**.
If the BST is empty, the new node becomes the root.

Question Explanation

In a Binary Search Tree:

- Left subtree contains values **smaller** than the node.
- Right subtree contains values **greater** than the node.

To insert a new value:

- Start from the root.
- Compare the key with the current node.
- Move **left** if the key is smaller.
- Move **right** if the key is greater.
- Insert the node at the first NULL position where it fits.

It is guaranteed that the key does **not already exist** in the BST.

Example Explanation

BST: 4 2 6 1 3 5 7
Insert key = 8

Traversal:

- 8 > 4 → move right
- 8 > 6 → move right
- 8 > 7 → move right → NULL

Insert 8 as right child of 7.

Algorithm

1. If root is NULL, create and return a new node.

2. Initialize a pointer `curr` starting from root.
3. Traverse the BST:
    - If key < `curr->val`, move to left subtree.
    - If key > `curr->val`, move to right subtree.
4. When a `NULL` position is found:
    - Insert the new node there.
5. Return the original root.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int key) {
        // If tree is empty, new node becomes root
        if (root == nullptr) {
            return new TreeNode(key);
        }

        TreeNode* curr = root;
```

```
        while (true) {
            if (key < curr->val) {
                if (curr->left != nullptr) {
                    curr = curr->left;
                } else {
                    curr->left = new TreeNode(key);
                    break;
                }
            } else {
                if (curr->right != nullptr) {
                    curr = curr->right;
                } else {
                    curr->right = new TreeNode(key);
                    break;
                }
            }
        }
        return root;
    }
};

// Inorder traversal to verify BST
void printInOrder(TreeNode* root) {
    if (!root) return;
    printInOrder(root->left);
    cout << root->val << " ";
    printInOrder(root->right);
}

int main() {
    TreeNode* root = new TreeNode(4);
    root->left = new TreeNode(2);
    root->right = new TreeNode(6);
    root->left->left = new TreeNode(1);
    root->left->right = new TreeNode(3);
    root->right->left = new TreeNode(5);
    root->right->right = new TreeNode(7);
```

```
    Solution sol;
    root = sol.insertIntoBST(root, 8);

    cout << "BST after insertion (Inorder): ";
    printInOrder(root);
    cout << endl;

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(H)`
  Where `H` is the height of the BST.
  - Balanced BST → `O(log N)`
  - Skewed BST → `O(N)`
- **Space Complexity:** `O(1)`
  Iterative approach uses constant extra space.

7. Delete a Node in Binary Search Tree

Given a **Binary Search Tree (BST)** and a key value, delete the node having that key from the BST.
After deletion, the BST property must remain valid.

Question Explanation

In a Binary Search Tree:

- Left subtree contains values **smaller** than the node.
- Right subtree contains values **greater** than the node.

While deleting a node, there are **three possible cases**:

1. **Node is a leaf (no children)**
   → Simply remove the node.
2. **Node has one child**
   → Replace the node with its child.
3. **Node has two children**
   → Replace the node's value with its **inorder successor** (smallest value in right subtree), (Inorder successor **ya** inorder predecessor — dono valid hote hain)
   → Then delete that successor node.

Example Explanation

BST: 5 3 6 2 4 -1 7
Delete key = 3

- Node 3 has two children (2 and 4)
- Inorder successor of 3 is 4
- Replace 3 with 4
- Delete original node 4

Final BST remains valid.

Algorithm

1. If root is NULL, return NULL.
2. If key < root value, delete from left subtree.
3. If key > root value, delete from right subtree.
4. If key == root value:
    - If left child is NULL, return right child.
    - If right child is NULL, return left child.
    - If both children exist:
        - Find inorder successor (minimum in right subtree).
        - Replace root value with successor value.
        - Delete successor from right subtree.
5. Return the root.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Definition of TreeNode
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Function to find minimum value node in BST
```

```
TreeNode* findMin(TreeNode* root) {
    while (root->left != nullptr) {
        root = root->left;
    }
    return root;
}

// Function to delete a node from BST
TreeNode* deleteNode(TreeNode* root, int key) {
    // Base case
    if (root == nullptr) return nullptr;

    // Traverse left subtree
    if (key < root->val) {
        root->left = deleteNode(root->left, key);
    }
    // Traverse right subtree
    else if (key > root->val) {
        root->right = deleteNode(root->right, key);
    }
    // Node found
    else {
        // Case 1: No left child
        if (root->left == nullptr) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        }
        // Case 2: No right child
        else if (root->right == nullptr) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        }
        // Case 3: Two children
        TreeNode* temp = findMin(root->right);
        root->val = temp->val;
        root->right = deleteNode(root->right, temp->val);
```

```
        }
        return root;
    }
};

// Inorder traversal to verify BST
void printInOrder(TreeNode* root) {
    if (!root) return;
    printInOrder(root->left);
    cout << root->val << " ";
    printInOrder(root->right);
}

int main() {
    TreeNode* root = new TreeNode(5);
    root->left = new TreeNode(3);
    root->right = new TreeNode(6);
    root->left->left = new TreeNode(2);
    root->left->right = new TreeNode(4);
    root->right->right = new TreeNode(7);

    Solution sol;
    root = sol.deleteNode(root, 3);

    cout << "BST after deletion (Inorder): ";
    printInOrder(root);
    cout << endl;

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(H)`
  Where `H` is the height of the BST.

- Balanced BST → `O(log N)`
- Skewed BST → `O(N)`

- **Space Complexity:** `O(H)`
  Due to recursion stack.
  - Balanced BST → `O(log N)`
  - Skewed BST → `O(N)`

8. Kth Largest / Smallest Element in Binary Search Tree

Given the root of a **Binary Search Tree (BST)** and an integer k, return:

- the **kth smallest** element
- the **kth largest** element

The result should be returned as an array where:

- first value = kth smallest
- second value = kth largest

The indexing is **1-based**.

Question Explanation

In a Binary Search Tree:

- **Inorder traversal** gives nodes in **sorted (ascending) order**
- **Reverse inorder traversal** gives nodes in **descending order**

Using this property, we can find the kth smallest and kth largest elements.

Example Explanation

BST: `[3, 1, 4, null, 2]`, `k = 1`

Inorder traversal → `[1, 2, 3, 4]`

- 1st smallest = 1
- 1st largest = 4

Output → `[1, 4]`

Approach 1: Brute Force

Algorithm

1. Create a vector to store node values.
2. Perform **inorder traversal** of the BST and store all values.
3. Since inorder traversal is sorted:
    - kth smallest = `values[k-1]`
    - kth largest = `values[n-k]`
4. Return both values.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Tree node structure
struct TreeNode {
    int data;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void inorder(TreeNode* root, vector<int>& vals) {
        if (!root) return;
        inorder(root->left, vals);
        vals.push_back(root->data);
        inorder(root->right, vals);
    }

    vector<int> kLargesSmall(TreeNode* root, int k) {
        vector<int> vals;
        inorder(root, vals);
        int kthSmallest = vals[k - 1];
        int kthLargest = vals[vals.size() - k];
        return {kthSmallest, kthLargest};
    }
};

int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(1);
    root->left->right = new TreeNode(2);
    root->right = new TreeNode(4);
```

```
    Solution sol;
    int k = 1;
    vector<int> ans = sol.kLargesSmall(root, k);

    cout << "[" << ans[0] << ", " << ans[1] << "]";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Every node is visited once during inorder traversal.

- **Space Complexity:** `O(N)`
  Extra space is used to store all node values.

Approach 2: Optimal (Without Extra Array)

Algorithm

Kth Smallest

1. Perform **inorder traversal**.
2. Maintain a counter.
3. When counter reaches k, store the value.

Kth Largest

1. Perform **reverse inorder traversal** (Right → Root → Left).
2. Maintain a counter.
3. When counter reaches k, store the value.

Return both results.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Tree node structure
struct TreeNode {
    int data;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int k, ans;

    void inorder(TreeNode* root) {
        if (!root || ans != -1) return;
        inorder(root->left);
        if (--k == 0) {
            ans = root->data;
            return;
        }
        inorder(root->right);
    }

    void reverseInorder(TreeNode* root) {
        if (!root || ans != -1) return;
        reverseInorder(root->right);
        if (--k == 0) {
            ans = root->data;
            return;
        }
        reverseInorder(root->left);
    }
```

```
    vector<int> kLargesSmall(TreeNode* root, int kVal) {
        k = kVal;
        ans = -1;
        inorder(root);
        int kthSmallest = ans;

        k = kVal;
        ans = -1;
        reverseInorder(root);
        int kthLargest = ans;

        return {kthSmallest, kthLargest};
    }
};

int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(1);
    root->left->right = new TreeNode(2);
    root->right = new TreeNode(4);

    Solution sol;
    int k = 1;
    vector<int> ans = sol.kLargesSmall(root, k);

    cout << "[" << ans[0] << ", " << ans[1] << "]";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  In worst case, traversal may visit all nodes.

- **Space Complexity:** `O(H)`
  Due to recursion stack, where `H` is the height of the BST.

- Balanced BST → `O(log N)`
- Skewed BST → `O(N)`

9. Check if a Tree is a Binary Search Tree (BST) or just a Binary Tree (BT)

Problem Explanation

You are given the root of a binary tree.
Your task is to **check whether the given tree follows the Binary Search Tree (BST) property or not**.

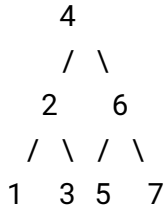
A **Binary Search Tree (BST)** has the following property:

- All values in the **left subtree** of a node are **strictly smaller** than the node's value.
- All values in the **right subtree** of a node are **strictly greater** than the node's value.
- This property must hold **for every node** in the tree.

If the tree satisfies this property, return **true**, otherwise return **false**.

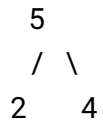
Example Explanation

Example 1 (BST):



Inorder traversal → 1 2 3 4 5 6 7 (sorted)
So, this tree **is a BST**.

Example 2 (Not a BST):



Here, 4 is on the right of 5 but 4 < 5, which violates BST rules.
So, this tree **is NOT a BST**.

Approach 1: Inorder Traversal Check

Algorithm

This approach is based on a key property of BST:

- **Inorder traversal of a BST always produces a strictly increasing sequence**

Steps:

1. Perform an inorder traversal of the binary tree.
2. Store the traversal values in a vector.
3. Check if the vector is **strictly sorted**.
4. If yes → tree is a BST.
5. Otherwise → tree is not a BST.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Tree node structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void inorder(TreeNode* root, vector<int>& arr) {
        if (!root) return;
        inorder(root->left, arr);
        arr.push_back(root->val);
        inorder(root->right, arr);
    }

    bool isBST(TreeNode* root) {
        vector<int> arr;
        inorder(root, arr);

        for (int i = 1; i < arr.size(); i++) {
            if (arr[i] <= arr[i - 1]) {
                return false;
            }
        }
        return true;
    }
};

int main() {
    TreeNode* root = new TreeNode(4);
    root->left = new TreeNode(2);
    root->right = new TreeNode(6);
    root->left->left = new TreeNode(1);
    root->left->right = new TreeNode(3);
```

```
    Solution sol;
    if (sol.isBST(root))
        cout << "Tree is a BST";
    else
        cout << "Tree is NOT a BST";

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each node is visited once during inorder traversal.

- **Space Complexity:** `O(N)`
  Extra space is used to store inorder traversal values.

Approach 2: Range (Min–Max) Validation (Optimal)

Algorithm

Instead of storing values, we validate each node using a **range**.

Steps:

1. For each node, maintain a valid range `(min, max)`.
2. Initially, range is `(-∞, +∞)`.
3. For every node:
    - Node value must lie strictly between `min` and `max`.
4. For left child:
    - New range → `(min, current node value)`

5. For right child:
    - New range → `(current node value, max)`
6. If any node violates the range, return false.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// Tree node structure
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    bool check(TreeNode* root, long minVal, long maxVal) {
        if (!root) return true;

        if (root->val <= minVal || root->val >= maxVal)
            return false;

        return check(root->left, minVal, root->val) &&
               check(root->right, root->val, maxVal);
    }

    bool isBST(TreeNode* root) {
        return check(root, LONG_MIN, LONG_MAX);
    }
};

int main() {
```

```
    TreeNode* root = new TreeNode(5);
    root->left = new TreeNode(2);
    root->right = new TreeNode(4);

    Solution sol;
    if (sol.isBST(root))
        cout << "Tree is a BST";
    else
        cout << "Tree is NOT a BST";

    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each node is checked once.

- **Space Complexity:** `O(H)`
  Due to recursion stack, where H is tree height.

    - Balanced tree → `O(log N)`

    - Skewed tree → `O(N)`

10. Lowest Common Ancestor in a Binary Search Tree

You are given the **root of a Binary Search Tree (BST)** and two nodes n1 and n2 that are guaranteed to be present in the tree.
Your task is to find the **Lowest Common Ancestor (LCA)** of these two nodes.

The **Lowest Common Ancestor** of two nodes is the **deepest node (farthest from the root)** that has **both nodes as its descendants**.
A node can also be a descendant of itself.

Because the tree is a **Binary Search Tree**, it follows:

- Left subtree values `< root`
- Right subtree values `> root`

We will use this property to optimize the solution.

Example Explanation

Example 1:

```
BST:
         8
        / \
       4   12
          /  \
         10  14
n1 = 4, n2 = 14
```

- Node 8 is the first node where one value lies on the left and the other on the right.
- Hence, **LCA = 8**

```
n1 = 10, n2 = 14
```

- Both nodes lie in the **right subtree of 8**
- At node 12, one node lies on the left and the other on the right
- Hence, **LCA = 12**

Approach 1: Naive Approach (Using Binary Tree Method)

Algorithm

This approach does **not use BST properties** and works for a normal binary tree.

1. If the current node is `NULL`, return `NULL`.
2. If the current node matches n1 or n2, return the current node.
3. Recursively find LCA in left subtree.
4. Recursively find LCA in right subtree.
5. If both left and right recursive calls return non-null values, current node is the LCA.
6. Otherwise, return the non-null value.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x) : data(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    Node* LCA(Node* root, Node* n1, Node* n2) {
        if (root == nullptr) return nullptr;

        if (root == n1 || root == n2)
            return root;

        Node* leftLCA = LCA(root->left, n1, n2);
```

```
        Node* rightLCA = LCA(root->right, n1, n2);

        if (leftLCA && rightLCA)
            return root;

        return (leftLCA != nullptr) ? leftLCA : rightLCA;
    }
};
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  In the worst case, all nodes are visited.

- **Space Complexity:** `O(N)`
  Due to recursion stack in a skewed tree.

Approach 2: Using BST Properties (Recursive Approach)

Algorithm

This approach uses the **ordering property of BST**.

1. If root is `NULL`, return `NULL`.

2. If both n1 and n2 are smaller than root, LCA lies in the **left subtree**.

3. If both n1 and n2 are greater than root, LCA lies in the **right subtree**.

4. Otherwise, the current root is the **LCA**.

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
```

```
    int data;
    Node* left;
    Node* right;
    Node(int x) : data(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    Node* LCA(Node* root, Node* n1, Node* n2) {
        if (root == nullptr)
            return nullptr;

        if (root->data > n1->data && root->data > n2->data)
            return LCA(root->left, n1, n2);

        if (root->data < n1->data && root->data < n2->data)
            return LCA(root->right, n1, n2);

        return root;
    }
};
```

- **Time Complexity:** `O(H)`
  We traverse only one path of the BST, where `H` is the height.

- **Space Complexity:** `O(H)`
  Due to recursion stack.

Approach 3: Using BST Properties (Iterative Method)

Algorithm

This is an optimized version of Approach 2 where recursion is removed.

1. Start from the root.

2. While root is not NULL:
    - If both nodes are smaller than root, move left.
    - If both nodes are greater than root, move right.
    - Otherwise, current node is the LCA.
3. Return the current node.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x) : data(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    Node* LCA(Node* root, Node* n1, Node* n2) {
        while (root != nullptr) {
            if (root->data > n1->data && root->data > n2->data)
                root = root->left;
            else if (root->data < n1->data && root->data < n2->data)
                root = root->right;
            else
                return root;
        }
        return nullptr;
    }
};
```

Complexity Analysis

- **Time Complexity:** `O(H)`
  Only one downward traversal of the BST.

- **Space Complexity:** `O(1)`
  No recursion or extra data structures used.

11. Construct BST from Preorder Traversal

Question Explanation

You are given the **preorder traversal** of a **Binary Search Tree (BST)**.
Your task is to **construct the BST** from this preorder traversal.

In a BST:

- All values in the left subtree are **smaller** than the root.
- All values in the right subtree are **greater** than the root.
- Preorder traversal follows the order: **Root → Left → Right**.

The constructed tree must be such that **its preorder traversal is exactly the given array**.

Example Explanation

Input:
```
pre = [10, 5, 1, 7, 40, 50]
```

- `10` is the first element → root

- Values smaller than `10` (`5, 1, 7`) go to left subtree
- Values greater than `10` (`40, 50`) go to right subtree
- Same logic is applied recursively

Inorder traversal of the constructed BST will be:
`1 5 7 10 40 50`

Approach 1: Naive – One by One Insert

Algorithm

1. Create an empty BST.
2. Traverse the preorder array from left to right.
3. Insert each element into the BST using normal BST insertion rules.
4. After all insertions, the BST is constructed.

This approach directly uses the property that inserting nodes one by one into a BST will form a valid BST.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int key;
    Node* left;
    Node* right;
    Node(int val) {
        key = val;
        left = nullptr;
        right = nullptr;
```

```
    }
};

// Standard BST insert function
Node* insertBST(Node* root, int key) {
    if (root == nullptr)
        return new Node(key);

    if (key < root->key)
        root->left = insertBST(root->left, key);
    else if (key > root->key)
        root->right = insertBST(root->right, key);

    return root;
}

// Construct BST from preorder traversal
Node* construct(vector<int>& pre) {
    Node* root = nullptr;
    for (int key : pre) {
        root = insertBST(root, key);
    }
    return root;
}

// Inorder traversal
void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->key << " ";
    inorder(root->right);
}

int main() {
    vector<int> pre = {10, 5, 1, 7, 40, 50};
    Node* root = construct(pre);
    inorder(root);
    return 0;
```

```
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  Each insertion can take $O(N)$ in the worst case (skewed BST).

- **Space Complexity:** $O(N)$
  Due to recursion stack in worst case.

Approach 2: Better – Find First Greater than Root

Algorithm

1. The first element of preorder is always the root.
2. Find the **first element greater than root**.
3. Elements between `(root index + 1)` and `(i - 1)` form the **left subtree**.
4. Elements from `i` to end form the **right subtree**.
5. Recursively apply the same logic to left and right subarrays.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x) {
```

```
        data = x;
        left = nullptr;
        right = nullptr;
    }
};

Node* constructUtil(vector<int>& pre, int low, int high) {
    if (low > high)
        return nullptr;

    Node* root = new Node(pre[low]);
    if (low == high)
        return root;

    int i;
    for (i = low + 1; i <= high; i++) {
        if (pre[i] > root->data)
            break;
    }

    root->left = constructUtil(pre, low + 1, i - 1);
    root->right = constructUtil(pre, i, high);

    return root;
}

Node* construct(vector<int>& pre) {
    return constructUtil(pre, 0, pre.size() - 1);
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main() {
```

```
    vector<int> pre = {10, 5, 1, 7, 40, 50};
    Node* root = construct(pre);
    inorder(root);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  For each node, we scan part of the array to find the split index.

- **Space Complexity:** $O(N)$
  Due to recursion stack.

Approach 3: Efficient – Pass Range in Recursion

Algorithm

This approach uses the **valid range of values** for each subtree.

1. Maintain an index `idx` for preorder traversal.
2. Start with range $(-\infty, +\infty)$.
3. If current value is not in range, return `NULL`.
4. Create node using current value and increment index.
5. Recur for:
    - Left subtree with range `(min, root->data)`
    - Right subtree with range `(root->data, max)`
6. Continue until all elements are used.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

Node* constructUtil(vector<int>& pre, int& idx, int minVal, int
maxVal) {
    if (idx >= pre.size())
        return nullptr;

    int key = pre[idx];
    if (key <= minVal || key >= maxVal)
        return nullptr;

    Node* root = new Node(key);
    idx++;

    root->left = constructUtil(pre, idx, minVal, key);
    root->right = constructUtil(pre, idx, key, maxVal);

    return root;
}

Node* constructTree(vector<int>& pre) {
    int idx = 0;
    return constructUtil(pre, idx, INT_MIN, INT_MAX);
```

```
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main() {
    vector<int> pre = {10, 5, 1, 7, 40, 50};
    Node* root = constructTree(pre);
    inorder(root);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each element is processed exactly once.

- **Space Complexity:** `O(N)`
  Recursion stack in worst case (skewed tree).

12. Inorder Successor / Predecessor in Binary Search Tree

You are given a **Binary Search Tree (BST)** and a **key value** representing a node in the tree.
Your task is to find:

- **Inorder Predecessor**:
  The node that appears **just before** the given node in inorder traversal.

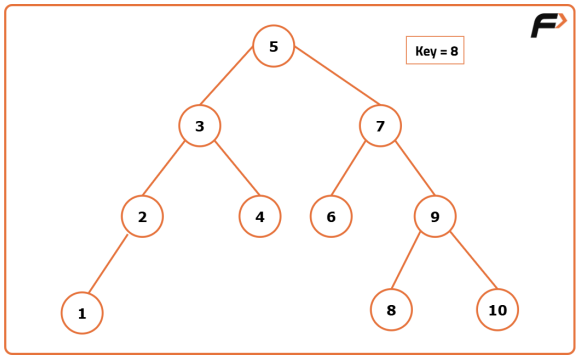
- **Inorder Successor**:
  The node that appears **just after** the given node in inorder traversal.

If no predecessor or successor exists, return `nullptr`.

Inorder traversal of BST always gives a **sorted sequence**.

Example Explanation

Input BST (inorder):
`1 2 3 4 5 6 7 8 9 10`



Key = 8

- Predecessor = 7
- Successor = 9

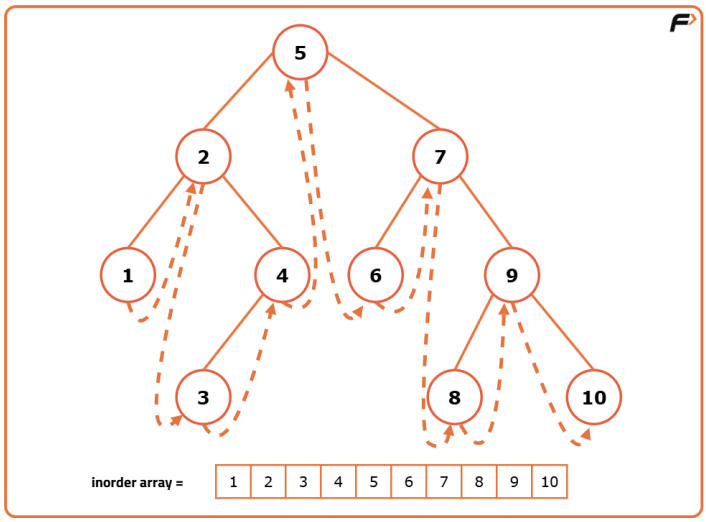
Approach 1: Brute Force

Algorithm

1. Perform **inorder traversal** of the BST and store all values in a vector.
2. Since inorder traversal of BST is sorted:

- Find the index of `key`.
- Predecessor = element at `index - 1` (if exists).
- Successor = element at `index + 1` (if exists).

3. If index is at boundary, return `nullptr` accordingly.



Code

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode{
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x): val(x), left(nullptr), right(nullptr) {}
};

class Solution{
public:
    void inorder(TreeNode* root, vector<int>& arr){
        if(!root) return;
        inorder(root->left, arr);
        arr.push_back(root->val);
```

```
        inorder(root->right, arr);
    }

    pair<int,int> findPreSuc(TreeNode* root, int key){
        vector<int> arr;
        inorder(root, arr);

        int pre = -1, suc = -1;
        for(int i=0;i<arr.size();i++){
            if(arr[i] == key){
                if(i > 0) pre = arr[i-1];
                if(i < arr.size()-1) suc = arr[i+1];
                break;
            }
        }
        return {pre, suc};
    }
};
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Entire tree is traversed.

- **Space Complexity:** `O(N)`
  Inorder array storage.

Approach 2: Better (Using BST Property – Iterative Traversal)

Algorithm

For Successor

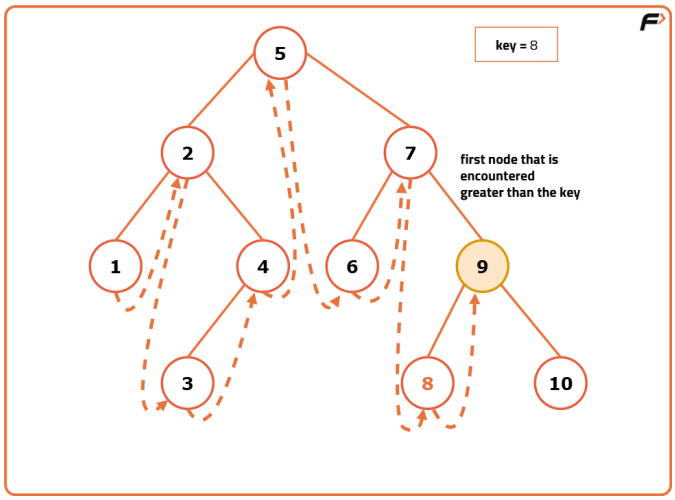
- Start from root.

- If current value > key:
    - Update successor.
    - Move left.
- Else move right.

For Predecessor

- Start from root.
- If current value < key:
    - Update predecessor.
    - Move right.
- Else move left.

Traversal continues until root becomes null.



Code

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode{
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x): val(x), left(nullptr), right(nullptr) {}
};

class Solution{
```

```
public:
    pair<TreeNode*,TreeNode*> findPreSuc(TreeNode* root, int key){
        TreeNode* pre = nullptr;
        TreeNode* suc = nullptr;
        TreeNode* cur = root;

        // find predecessor
        while(cur){
            if(cur->val < key){
                pre = cur;
                cur = cur->right;
            } else {
                cur = cur->left;
            }
        }

        cur = root;
        // find successor
        while(cur){
            if(cur->val > key){
                suc = cur;
                cur = cur->left;
            } else {
                cur = cur->right;
            }
        }

        return {pre, suc};
    }
};
```

Complexity Analysis

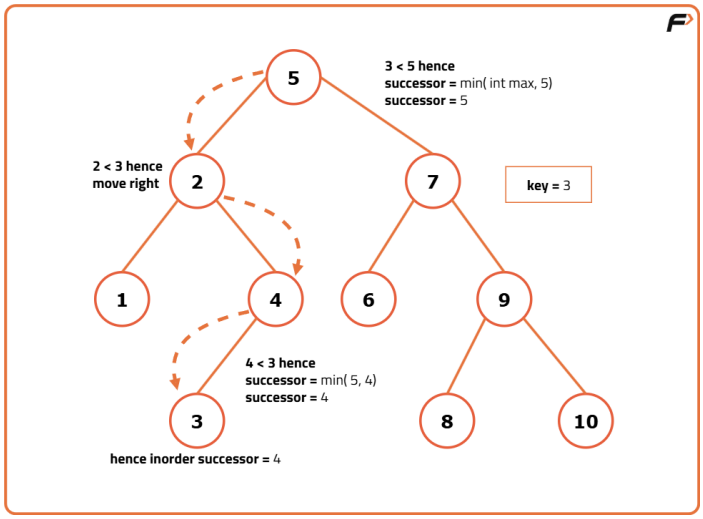
- **Time Complexity:** $O(H)$
  Only height of BST is traversed.

- **Space Complexity:** $O(1)$
  No extra data structures used.

Approach 3: Optimal (BST Property – Single Traversal)

Algorithm

1. Traverse BST once.
2. If node value equals key:
   - Stop.
3. If node value > key:
   - Update successor.
   - Move left.
4. If node value < key:
   - Update predecessor.
   - Move right.
5. Continue until null.



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct TreeNode{
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x): val(x), left(nullptr), right(nullptr) {}
};

class Solution{
public:
    pair<TreeNode*,TreeNode*> findPreSuc(TreeNode* root, int key){
        TreeNode* pre = nullptr;
        TreeNode* suc = nullptr;

        while(root){
            if(root->val == key){
                break;
            }
            else if(root->val > key){
                suc = root;
                root = root->left;
            }
            else{
                pre = root;
                root = root->right;
            }
        }
        return {pre, suc};
    }
};
```

- **Time Complexity:** $O(H)$
  Only one downward traversal.

- **Space Complexity:** $O(1)$
  Constant extra space.

13. Merge 2 BSTs

You are given **two Binary Search Trees (BSTs)**.
Your task is to return **all elements from both BSTs in sorted order**.

Remember:

- Inorder traversal of a BST gives elements in **sorted order**.
- Duplicate values are allowed and should be included.

Example Explanation

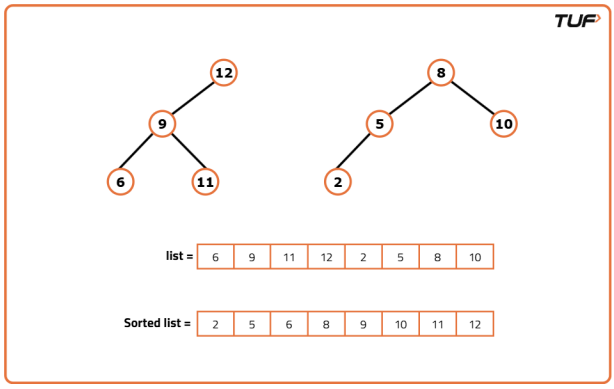
Input
BST 1: `[5, 3, 6, 2, 4]`
BST 2: `[2, 1, 3, null, null, null, 7, 6]`

After collecting and sorting all elements:
`[1, 2, 2, 3, 3, 4, 5, 6, 6, 7]`

Approach 1: Brute Force

1. Traverse the first BST and store all its elements in a list.
2. Traverse the second BST and store all its elements in the same list.
3. Sort the combined list.
4. Return the sorted list.

This approach ignores the BST property during traversal and sorts later.



Code

```
#include <bits/stdc++.h>
using namespace std;

// Structure for BST node
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    void traverse(Node* root, vector<int>& arr) {
        if (!root) return;
        traverse(root->left, arr);
        arr.push_back(root->data);
        traverse(root->right, arr);
    }

    vector<int> mergeBSTs(Node* root1, Node* root2) {
        vector<int> arr;
        traverse(root1, arr);
        traverse(root2, arr);
        sort(arr.begin(), arr.end());
        return arr;
    }
```

```
};

int main() {
    Node* root1 = new Node(2);
    root1->left = new Node(1);
    root1->right = new Node(4);

    Node* root2 = new Node(3);
    root2->left = new Node(0);
    root2->right = new Node(5);

    Solution sol;
    vector<int> result = sol.mergeBSTs(root1, root2);

    for (int x : result) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O((n + m) log(n + m))`
  Sorting dominates the time.

- **Space Complexity:** `O(n + m)`
  Extra list to store all elements.

Approach 2: Optimal (Using Inorder Traversal + Merge)

1. Perform **inorder traversal** on the first BST and store elements in `arr1`.

2. Perform **inorder traversal** on the second BST and store elements in `arr2`.

    - Both arrays will already be sorted.

3. Merge the two sorted arrays using the **two-pointer technique**.

4. Return the merged sorted array.

This approach fully uses BST properties.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Structure for BST node
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    void inorder(Node* root, vector<int>& arr) {
        if (!root) return;
        inorder(root->left, arr);
        arr.push_back(root->data);
        inorder(root->right, arr);
    }

    vector<int> mergeArrays(vector<int>& a, vector<int>& b) {
        vector<int> res;
        int i = 0, j = 0;

        while (i < a.size() && j < b.size()) {
            if (a[i] < b[j]) res.push_back(a[i++]);
            else res.push_back(b[j++]);
        }

        while (i < a.size()) res.push_back(a[i++]);
        while (j < b.size()) res.push_back(b[j++]);

        return res;
    }
```

```
    vector<int> mergeBSTs(Node* root1, Node* root2) {
        vector<int> arr1, arr2;
        inorder(root1, arr1);
        inorder(root2, arr2);
        return mergeArrays(arr1, arr2);
    }
};

int main() {
    Node* root1 = new Node(3);
    root1->left = new Node(1);
    root1->right = new Node(5);

    Node* root2 = new Node(4);
    root2->left = new Node(2);
    root2->right = new Node(6);

    Solution sol;
    vector<int> result = sol.mergeBSTs(root1, root2);

    for (int x : result) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n + m)`
  Each node is visited once, merge is linear.

- **Space Complexity:** `O(n + m)`
  Two inorder arrays and merged result.

14. Two Sum In BST | Check if there exists a pair with Sum K

You are given the **root of a Binary Search Tree (BST)** and an integer **K**.
You need to check whether there exist **two different nodes** in the BST such that the **sum of their values is exactly K**.

Return **true** if such a pair exists, otherwise return **false**.

Because this is a BST, its **inorder traversal gives a sorted sequence**, which can be used to optimize the solution.

Example Explanation

Example 1

BST: 5 3 6 2 4 -1 7, K = 9

Inorder traversal → [2, 3, 4, 5, 6, 7]
Possible pair: 2 + 7 = 9
So, output is **true**.

Approach 1: Brute Force (Inorder + Two Pointers)

Algorithm

1. Perform an **inorder traversal** of the BST and store all node values in a vector.
2. Since inorder traversal of a BST is sorted, apply the **two pointer technique**:
    - One pointer at the start.
    - One pointer at the end.
3. While left pointer is smaller than right pointer:

- If `arr[left] + arr[right] == K`, return true.
- If sum is smaller than K, move left pointer forward.
- If sum is greater than K, move right pointer backward.

4. If no such pair is found, return false.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    bool findTarget(TreeNode* root, int k) {
        vector<int> inorder;
        inorderTraversal(root, inorder);

        int l = 0, r = inorder.size() - 1;
        while (l < r) {
            int sum = inorder[l] + inorder[r];
            if (sum == k) return true;
            else if (sum < k) l++;
            else r--;
        }
        return false;
    }

private:
    void inorderTraversal(TreeNode* root, vector<int>& inorder) {
```

```
        if (!root) return;
        inorderTraversal(root->left, inorder);
        inorder.push_back(root->val);
        inorderTraversal(root->right, inorder);
    }
};

int main() {
    TreeNode* root = new TreeNode(5);
    root->left = new TreeNode(3);
    root->right = new TreeNode(6);
    root->left->left = new TreeNode(2);
    root->left->right = new TreeNode(4);
    root->right->right = new TreeNode(7);

    Solution sol;
    int k = 9;
    cout << sol.findTarget(root, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Inorder traversal takes O(N) and two-pointer scan also takes O(N).

- **Space Complexity:** O(N)
  Extra space is used to store inorder traversal.

Approach 2: Optimal (BST Iterator – Two Pointer Without Extra Array)

Algorithm

This approach avoids storing the entire inorder traversal.

1. Create two iterators:
    - One inorder iterator (smallest to largest).
    - One reverse inorder iterator (largest to smallest).
2. Initialize two values:
    - `i` from the inorder iterator.
    - `j` from the reverse inorder iterator.
3. While `i < j`:
    - If `i + j == K`, return true.
    - If sum is smaller than K, move inorder iterator forward.
    - If sum is greater than K, move reverse inorder iterator backward.
4. If no pair is found, return false.

This simulates the two-pointer technique directly on the BST.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// push leftmost path (normal inorder)
void pushLeft(TreeNode* node, stack<TreeNode*>& st) {
    while (node) {
```

```
        st.push(node);
        node = node->left;
    }
}

// push rightmost path (reverse inorder)
void pushRight(TreeNode* node, stack<TreeNode*>& st) {
    while (node) {
        st.push(node);
        node = node->right;
    }
}

bool findTarget(TreeNode* root, int k) {
    if (!root) return false;

    stack<TreeNode*> stL, stR;

    // initialize both stacks
    pushLeft(root, stL);   // smallest
    pushRight(root, stR);  // largest

    while (!stL.empty() && !stR.empty()) {
        TreeNode* leftNode = stL.top();
        TreeNode* rightNode = stR.top();

        // IMPORTANT: same node use nahi karna
        if (leftNode == rightNode) break;

        int sum = leftNode->val + rightNode->val;

        if (sum == k)
            return true;
        else if (sum < k) {
            // move left pointer forward
            stL.pop();
            pushLeft(leftNode->right, stL);
        } else {
```

```
            // move right pointer backward
            stR.pop();
            pushRight(rightNode->left, stR);
           //pushLeft() function khud NULL handle karta hai
        }
    }
    return false;
}

int main() {
    TreeNode* root = new TreeNode(7);
    root->left = new TreeNode(3);
    root->right = new TreeNode(15);
    root->right->left = new TreeNode(9);
    root->right->right = new TreeNode(20);

    int k = 22;
    cout << findTarget(root, k);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each node is visited at most once by the iterators.

- **Space Complexity:** O(H)
  Stack space used by iterators, where H is the height of the BST.

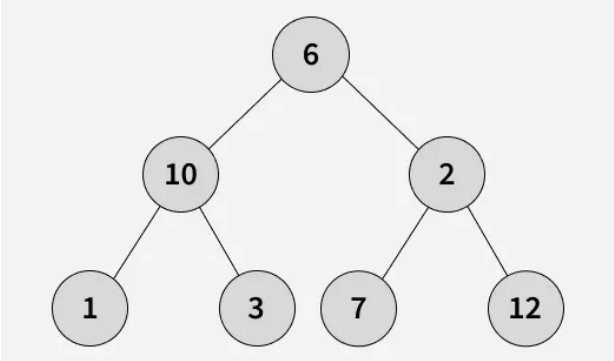
15. Recover BST – Two Nodes are Swapped, Correct it

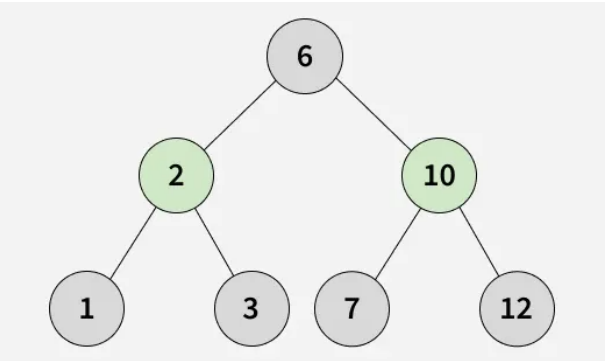
You are given a **Binary Search Tree (BST)** in which **exactly two nodes have been swapped by mistake**.
Because of this swap, the tree no longer follows the BST property.

Your task is to **restore the BST** by correcting the swapped nodes **without changing the tree structure**.

A valid BST has the property that its **inorder traversal is always sorted**.

Example Explanation





If inorder traversal of a BST should be:

```
[1, 2, 3, 6, 7, 10, 12]
```

But due to swapping, it becomes:

```
[1, 6, 3, 2, 7, 10, 12]
```

Here, **two values break the sorted order**, which helps us identify the swapped nodes.

Approach 1: Inorder Traversal + Sorting (Naive)

1. Perform **inorder traversal** of the BST and store all node values in a vector.

2. Since exactly two nodes are swapped, the inorder array will not be sorted.
3. **Sort the inorder array** to get the correct order.
4. Perform inorder traversal again and **replace node values** using the sorted array.
5. The BST property is restored.

This approach fixes values, not structure.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x) : data(x), left(nullptr), right(nullptr) {}
};

void findInorder(Node* root, vector<int>& inorder) {
    if (!root) return;
    findInorder(root->left, inorder);
    inorder.push_back(root->data);
    findInorder(root->right, inorder);
}

void correctBSTUtil(Node* root, vector<int>& inorder, int& index) {
    if (!root) return;
    correctBSTUtil(root->left, inorder, index);
    root->data = inorder[index++];
    correctBSTUtil(root->right, inorder, index);
}

void correctBST(Node* root) {
    vector<int> inorder;
```

```
    findInorder(root, inorder);
    sort(inorder.begin(), inorder.end());
    int index = 0;
    correctBSTUtil(root, inorder, index);
}
```

Complexity Analysis

- **Time Complexity:** O(N log N)
  Inorder traversal is O(N) and sorting is O(N log N).

- **Space Complexity:** O(N)
  Extra array is used to store inorder traversal.

Approach 2: One Inorder Traversal (Expected / Optimal)

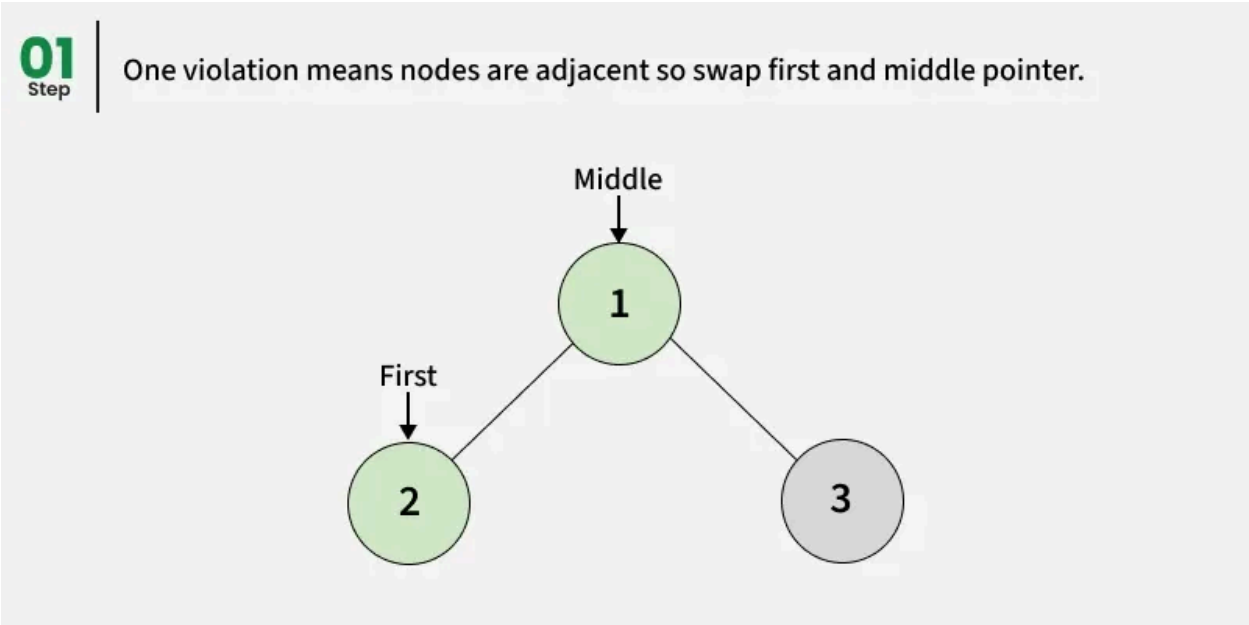
Algorithm

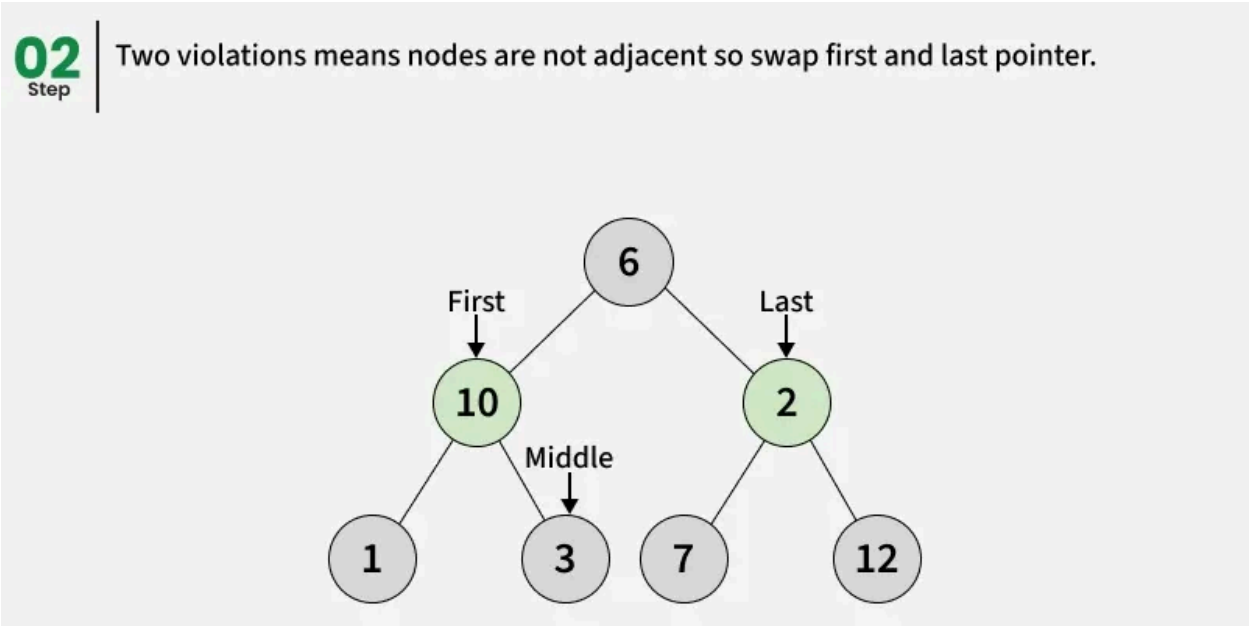
This approach uses the fact that inorder traversal of a BST should be sorted.

There are **two cases**:

1. **Swapped nodes are adjacent** in inorder traversal
   → Only one violation is found.

2. **Swapped nodes are not adjacent**
   → Two violations are found.





Steps:

1. Do an inorder traversal while tracking:
    - `prev` → previously visited node
    - `first`, `middle`, `last` → nodes involved in violation
2. Whenever `prev->data > current->data`, a violation is found.
3. On first violation:
    - `first = prev`
    - `middle = current`

4. On second violation:
    - `last = current`
5. After traversal:
    - If `last` exists → swap `first` and `last`
    - Else → swap `first` and `middle`

```
Node* first;   // pehla galat node

Node* middle;  // first violation ka current node

Node* last;    // second violation ka current node

Node* prev;    // inorder me previous node


Only one violation:

first = 3

middle = 2

last = NULL


Final pointers state

first  = 10

middle = 3

last   = 2
```

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int x) : data(x), left(nullptr), right(nullptr) {}
};

void correctBSTUtil(Node* root, Node*& first, Node*& middle,
                    Node*& last, Node*& prev) {
    if (!root) return;

    correctBSTUtil(root->left, first, middle, last, prev);

    if (prev && root->data < prev->data) {
        if (!first) {
            first = prev;
            middle = root;
        } else {
            last = root;
        }
    }

    prev = root;

    correctBSTUtil(root->right, first, middle, last, prev);
}

void correctBST(Node* root) {
    Node *first = nullptr, *middle = nullptr;
    Node *last = nullptr, *prev = nullptr;

    correctBSTUtil(root, first, middle, last, prev);

    if (first && last)
        swap(first->data, last->data);
    else if (first && middle)
        swap(first->data, middle->data);
}
```

Complexity Analysis

- **Time Complexity:** O(N)
  Single inorder traversal of the tree.

- **Space Complexity:** O(H)
  Recursive stack space, where H is the height of the BST.

16. Largest BST in Binary Tree

Given the root of a **Binary Tree**, return the **size of the largest subtree** which is also a **Binary Search Tree (BST)**.

A subtree is considered a BST if:

- Left subtree contains values **less than** root
- Right subtree contains values **greater than** root
- Both left and right subtrees are BSTs

Brute Force Approach

Algorithm

1. For every node in the binary tree, consider it as a potential root of a BST.
2. Check whether the subtree rooted at that node is a valid BST using range constraints.
3. If valid, compute the size of that subtree.
4. Maintain a global maximum size.

5. Return the maximum size found.

This approach repeatedly validates subtrees, leading to inefficiency.

Code (Brute Force)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

struct Info {
    int size;
    bool isBST;
    int minVal;
    int maxVal;
};

class Solution {
public:
    Info solve(TreeNode* node) {
        // Base case
        if (!node) {
            return {0, true, INT_MAX, INT_MIN};
        }

        Info left = solve(node->left);
        Info right = solve(node->right);

        // If current subtree is BST
        if (left.isBST && right.isBST &&
            left.maxVal < node->data &&
```

```
            node->data < right.minVal) {

            return {
                left.size + right.size + 1,   // size
                true,                          // isBST
                min(node->data, left.minVal), // min
                max(node->data, right.maxVal) // max
            };
        }

        // If not BST
        return {
            max(left.size, right.size),
            false,
            INT_MIN,
            INT_MAX
        };
    }

    int largestBST(TreeNode* root) {
        return solve(root).size;
    }
};
```

Complexity Analysis

- **Time Complexity:** $O(N^2)$
  Each node may validate an entire subtree.

- **Space Complexity:** O(H)
  Recursive stack where H is height of tree.

Optimal Approach (Postorder DP)

For each node, we only need **information from left and right subtrees**:

- Minimum value
- Maximum value
- Size of largest BST

Using **postorder traversal**, we compute everything bottom-up in **one pass**.

Algorithm

For every node, maintain:

- `minNode` → minimum value in subtree
- `maxNode` → maximum value in subtree
- `maxSize` → size of largest BST in subtree

Steps:

1. Recursively compute values for left and right subtrees.
2. If `left.maxNode < root.data < right.minNode`, subtree is BST:
    - Update size = left.size + right.size + 1
3. Else:
    - Mark subtree invalid using extreme values
    - Carry forward max size from children
4. Return the maximum size found.

Code (Optimal)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    struct NodeValue {
        int minNode, maxNode, maxSize;
        NodeValue(int minNode, int maxNode, int maxSize)
            : minNode(minNode), maxNode(maxNode), maxSize(maxSize) {}
    };

    NodeValue helper(TreeNode* node) {
        if (!node) {
            return NodeValue(INT_MAX, INT_MIN, 0);
        }

        NodeValue left = helper(node->left);
        NodeValue right = helper(node->right);

        if (left.maxNode < node->data && node->data < right.minNode) {
            return NodeValue(
                min(node->data, left.minNode),
                max(node->data, right.maxNode),
                left.maxSize + right.maxSize + 1
            );
        }

        return NodeValue(INT_MIN, INT_MAX, max(left.maxSize,
right.maxSize));
    }
```

```
    int largestBST(TreeNode* root) {
        return helper(root).maxSize;
    }
};
```

Complexity Analysis

- **Time Complexity:** O(N)
  Each node is processed exactly once.

- **Space Complexity:** O(H)
  Recursive stack space (height of tree).